SSG Auto Parts

Move code to GitHub and host the web pilot on Vercel · Which machine does what

PC A = this development PC (GitHub push blocked). **Laptop B** = machine that can reach GitHub. **Browser** = Vercel or Supabase dashboard (works on either PC). Supabase stays one cloud project; Vercel only hosts the static Expo web app.

Overview

PC A creates a zip → you transfer it → Laptop B commits and pushes to GitHub → Vercel builds from GitHub → testers use the Vercel URL. After that, PC A can still run Supabase and Vercel tasks in the browser without pushing code.

Phase 1 — Prepare zip (PC A only)

Step	Where	What to do
1.1	PC A	Confirm secrets are **not** in the folder you zip: **.env.local**, **Pass.txt**, **supabase/.temp/**.
1.2	PC A	Create the zip (includes **.git** history). Default output: **F:\Cursor\SSG-POC\ssg-autoparts-for-github.zip** (~29 MB).
1.3	PC A → Laptop B	Send the zip by email attachment or USB. Send **.env.local** values separately (password manager or USB only — not inside the zip).

PowerShell on PC A (recreate zip anytime):

```
Set-Location F:\Cursor\SSG-POC
tar -acf ssg-autoparts-for-github.zip `
  --exclude="ssg-autoparts/node_modules" `
  --exclude="ssg-autoparts/.expo" `
  --exclude="ssg-autoparts/dist" `
  --exclude="ssg-autoparts/web-build" `
  --exclude="ssg-autoparts/.env.local" `
  --exclude="ssg-autoparts/Pass.txt" `
  --exclude="ssg-autoparts/supabase/.temp" `
  ssg-autoparts
```

Included in zip vs excluded

Step	Where	What to do
✓	In zip	app/, components/, lib/, types/, supabase/migrations, supabase/functions, docs/, package.json, .env.local.example, .git/
✗	Excluded	node_modules (run npm ci on B), .expo, dist, .env.local, Pass.txt, supabase/.temp

Phase 2 — Unpack and local setup (Laptop B)

Step	Where	What to do
2.1	Laptop B	Unzip to a folder named **ssg-autoparts** with **package.json** at the root.
2.2	Laptop B	Copy **.env.local.example** to **.env.local**. Fill **EXPO_PUBLIC_SUPABASE_URL** and **EXPO_PUBLIC_SUPABASE_KEY** (publishable key only). Set **EXPO_PUBLIC_AUTH_MODE=dev**.
2.3	Laptop B	Install dependencies and optional web build smoke test.

Terminal on Laptop B:

```
cd ssg-autoparts
npm ci
npx expo export -p web
```

Phase 3 — Git commit and push (Laptop B)

Step	Where	What to do
3.1	Browser (B)	GitHub → New repository → **empty** (no README if you have local commits). Copy the HTTPS URL.
3.2	Laptop B	Stage source. Never add Pass.txt or .env.local.
3.3	Laptop B	Commit, add remote **origin**, push.

Terminal on Laptop B:

```
cd ssg-autoparts
git status

git add app components lib types supabase docs AGENTS.md
git add -u

git add components/CatalogueFilterBar.tsx components/CatalogueHierarchyPicker.tsx
git add components/PartListRow.tsx components/PreferenceMultiPick.tsx
git add lib/catalogue-preference-sort.ts lib/part-display.ts
git add lib/queries/catalogue-filters.ts lib/queries/catalogue-hierarchy.ts
git add supabase/migrations/010_client_itemlist_catalogue.sql
git add supabase/migrations/011_hierarchical_catalogue.sql
```

```
git add supabase/migrations/012_profile_catalogue_preferences.sql

git status
git commit -m "Hierarchical catalogue browse, onboarding preferences, and Excel import sync."

git remote add origin https://github.com/YOUR_ORG/ssg-autoparts.git
git push -u origin master
```

If GitHub uses **main**: run **git branch -M main** before push, then **git push -u origin main**. If **origin** already exists: **git remote set-url origin …** then push.

Phase 4 — Vercel (Browser, once)

Step	Where	What to do
4.1	Browser	vercel.com → Log in with GitHub → **Add New Project** → import **ssg-autoparts**.
4.2	Browser	Build settings: Install **npm ci**. Build **npx expo export -p web**. Output directory **dist**.
4.3	Browser	Environment variables (Production): same names as .env.local — EXPO_PUBLIC_SUPABASE_URL, EXPO_PUBLIC_SUPABASE_KEY, EXPO_PUBLIC_AUTH_MODE, optional EXPO_PUBLIC_SSG_WHATSAPP and EXPO_PUBLIC_SSG_PHONE.
4.4	Browser	Deploy. Note the URL (e.g. https://your-project.vercel.app). Every future **git push** to the connected branch redeploys automatically.
4.5	Browser	Open the Vercel URL → sign in as a test buyer → browse → cart → submit quote once.

Phase 5 — Supabase (Browser on A or B; CLI optional)

Step	Where	What to do
5.1	Browser	Ensure project is **active** (not paused). Catalogue imported. Migrations 001–012 applied on the linked project.
5.2	Browser	Authentication → Email on, confirm email off for pilot. Hook **Before user created** → hook_restrict_signup.
5.3	Browser	allowed_users + Auth users for each tester. Share passwords privately.
5.4	PC A or B	Optional CLI: link project and deploy functions if code changed.

Optional terminal (PC A or Laptop B):

```
supabase login
supabase link --project-ref YOUR_PROJECT_REF
supabase db push
supabase functions deploy import-parts
supabase functions deploy notify-quote
```

Project ref: Supabase → Project Settings → General. Edge function secrets (Resend, etc.) live in the Supabase dashboard, not in Git.

Phase 6 — Day-to-day after go-live

Step	Where	What to do
Code change	Laptop B (or A if push works later)	Edit → commit → **git push** → Vercel auto-rebuilds.
New tester	Browser (A or B)	Supabase: row in allowed_users + Auth user. No Vercel change.
Env var change	Browser	Vercel → Settings → Environment Variables → Redeploy.
Catalogue import	Browser	Admin signs in on the **Vercel URL** → admin upload (Edge Function).
PC A blocked on Git	PC A → B	Copy changed files or new zip → Laptop B commits and pushes.

What you do not need for web pilot

Apple Developer, Google Play, or EAS store builds — only for native app store later. Vercel Hobby is enough for a private pilot link; Supabase Free works for few testers (Pro avoids project pause during business testing).

See also: docs/GITHUB-UPLOAD.md in the repo. SSG Auto Parts · Trusted since 1971